JEV Decision Harness — guía para el equipo

Cómo usamos Jev para **construir código** en este repo, cómo se instala en local y qué decide cada actor (Jev vs el LLM del agente).

1. Qué es (y qué NO es)

El **JEV harness** es una capa de decisiones acotadas para el proceso de desarrollo: los agentes de código (codex, opencode, devin, claude, cursor) lo consultan en **momentos de decisión puntuales** mientras programan.

Jev no escribe código. Es una API de decisiones que responde con tipos cerrados:

Tipo	Significado	Ejemplo
`noul`	probabilidad 0–1	"¿qué tan probable es que esto sea destructivo?" → 0.97
`choice`	elige entre opciones dadas	`read_only / local_edit / destructive / external / other`
`score`	puntúa candidatos dados	archivo A: 0.9, archivo B: 0.2

El LLM del agente sigue haciendo **todo** el trabajo creativo: elegir archivos, escribir líneas, diseñar. Jev **juzga, clasifica y aconseja** — nunca genera.

2. Dos sistemas de decisión (no confundir)

	A — Dev harness (este doc)	B — Producción
Para qué	agentes codeando	el bot de WhatsApp en runtime
Código	`src/lib/system-one/{provider,packs,ledger,harness}` + `harness/`	`src/lib/system-one/{hosted,config,registry,index}.ts`
Entrada	packs + MCP + hooks	`decideForTurn()` por tenant → `/v1/decisions`
Config	`.env.local` (`JEV_API_KEY` , `DECISION_ENGINE`)	columnas `agent_config.decision_*` por tenant

Son independientes. Que el harness funcione **no** implica tráfico de producción pasando por Jev.

3. Cuándo se usa Jev — los momentos de decisión

Momento	Disparador	Pack(s)	Efecto
Antes de una tool mutadora	hook `PreToolUse` (codex) / plugin `tool.execute.before` (opencode)	`guard`	`deny/ask/allow` — el LLM propone, Jev **veta**

Cada fase SDD	`JEV_HARNESS_SDD=1` + `shadow-wiring.mts`	ver §5	**shadow**: mide y persiste, no actúa
Consulta voluntaria	tools MCP `jev_*` (14)	cualquiera	segunda opinión acotada
Antes de parar (opc.)	hook `Stop`	`verify` + checks.sh	bloquea el stop si la evidencia no alcanza
Despertar agente (opc.)	cron `wake.mjs`	`triage`	¿vale levantar un agente por este evento?

4. Qué decide cada uno

Decisión	Quién
Escribir código, elegir líneas, diseñar solución	**LLM** — siempre
Inventar la lista de archivos candidatos	LLM
¿Este comando es destructivo?	**Jev** (`guard`) — hard rules + 5-way risk
¿Cuáles de *estos* candidatos aplican?	Jev (`file-scope` , `test-impact` , `skill-route`)
¿Es ambiguo el pedido?	Jev (`ambiguity` , noul)
¿Qué tan grande es el impacto?	Jev (`blast-radius` , `workload-forecast`)
¿La evidencia cubre el objetivo?	Jev (`verify` , `acceptance` , `triage`)

> **Ejemplo real** (verificado en vivo, 12-oct):
>
> ```
> rm -rf /                                → deny  (hard_rule: rm-rf-root — código, sin modelo)
> git push origin main --delete-old-branches → deny  (Jev: destructive, above_threshold)
> git status                              → ask   (confianza bajo la banda → pide confirmar)
> ```

5. Catálogo de packs y fases SDD

`harness/sdd/shadow-wiring.mts` corre estos packs por fase:

Fase SDD	Packs
orchestrator	`skill-route` , `context-prime`
explore	`file-scope` , `ambiguity`
design	`file-scope` , `blast-radius`
tasks	`workload-forecast`

apply	`guard`, `scope-guard`, `test-impact`
verify	`acceptance`, `verify`, `triage`

Cada pack define preguntas tipadas (noul/choice/score), umbrales en `src/lib/system-one/policy.ts` y postura: **closed** (protectivo, fail-closed) u **open** (advisory, fail-open).

6. El Gate PreToolUse en detalle

`harness/hooks/gate-pretool.mjs` corre el pack `guard` antes de cada tool mutadora (`bash`, `write`, `edit`, `patch`, `multiedit`, `exec`…):

1. **Hard rules en código primero** — `rm -rf /`, `git push --force`, pipe a `sh`/`bash` → deny sin consultar al modelo.
2. **Jev clasifica el riesgo** en 5 categorías.
3. **Mapeo**: `destructive` → deny · `external` → ask · `read_only`/`local_edit` con confianza ≥ banda → allow · resto → ask.

Posturas de fallo (diseño deliberado):

Situación	Resultado
Sin provider (`rules`, sin key)	hard-rules-only; permite el resto (usable offline)
Provider configurado pero falla	ask (fail-closed — nunca allow ciego)
Hard rule matchea	deny autoritativo, aunque Jev esté caído

7. Shadow → ledger → promoción

Los packs **nacen en shadow**: corren, se miden y se persisten, pero nada actúa sobre su respuesta. El `ShadowLedger` guarda `DecisionSample`s en Engram bajo `sdd/<change>/decisions/<phase>` (provider, latencia, tokens, costo).

`scripts/jev-report.ts` (baseline vs +Jev A/B) es el gate de promoción: cuando los samples reales muestren que un pack gana su lugar, pasa de shadow a activo. Hasta entonces, **advisory only**.

> Estado honesto: hoy el único path que **actúa** sobre la respuesta es el Gate (§6). El resto es medición.

8. Instalación local

Prerequisitos: Node + `npm install`, Git Bash en PATH (Windows), y `.env.local` con:

```
JEV_API_KEY=ts_...            # key de Jev (server-side, nunca en configs)
DECISION_ENGINE=jev
DECISION_PROVIDER=jev
JEV_HARNESS_SDD=1             # habilita packs shadow en fases SDD
JEV_MEMORY=local              # local|engram | cloud | neon | none
ENGRAM_HTTP_URL=http://127.0.0.1:7437
JEV_MEMORY_PROJECT=whatsapp-agent-base
```

Paso 1 — skill `jev-decide` (le enseña al agente CUÁNDO consultar):

```
bash harness/install.sh --agent codex      # → .agents/skills/
bash harness/install.sh --agent opencode   # → .opencode/skills/
# opcional: --global para todos los proyectos
```

Paso 2 — MCP server `jev-harness` (las 14 tools `jev_*`). El wrapper carga `.env.local` — la key nunca va en el config:

```
bash -lc 'cd "<REPO>" && set -a && . ./.env.local 2>/dev/null; set +a; exec node harness/mcp/server.mjs'
```

Agente	Dónde registrar
opencode	`opencode.json` → `mcp.jev-harness` (ya está en este repo)
codex	`~/.codex/config.toml` → `[mcp_servers.jev-harness]`
devin	`mcp_config.json` → `mcpServers.jev-harness`
claude	`claude mcp add jev-harness -- bash -lc '<wrapper>'`
cursor	`.cursor/mcp.json`

Paso 3 — el Gate (consumo automático real):

Agente	Instalación
opencode	`opencode.json` → `"plugin": ["file:///<REPO>/harness/hooks/opencode-plugin.mjs"]`
codex	`~/.codex/hooks.json` → entrada `PreToolUse` con `command: jev-gate.cmd` (wrapper → `gate-pretool.mjs`)
claude code	mergear `harness/hooks/claude-hooks.json` en `.claude/settings.json`
devin	sin hook local — usa las tools MCP `jev_*`

Matcher recomendado (codex): `Bash|Write|Edit|MultiEdit|Patch|apply_patch|exec|exec_command|shell`

Paso 4 — verificar:

```
# tools MCP (14)
printf '%s\n' '{"jsonrpc":"2.0","id":1,"method":"tools/list"}' | node harness/mcp/server.mjs

# gate en vivo (con env cargado)
node harness/hooks/gate-pretool.mjs --tool Bash --state "rm -rf /"    # → deny
node harness/hooks/gate-pretool.mjs --tool Bash --state "git status"  # → allow|ask (Jev real)

# shadow packs por fase SDD
npx tsx harness/sdd/shadow-wiring.mts --change demo --phase apply --task "test"

# tests
npm test                                                          # 155 tests TS
node --test harness/hooks/hooks.test.mjs harness/mcp/mcp.test.mjs
```

`engine=jev` en la salida del shadow-wiring = la key está pegando a la API real; `engine=rules` = cayó a fallback (revisar key/red).

9. Estado actual + gaps conocidos

- **Instalado y verificado en vivo**: gate en codex + plugin en opencode + MCP en los tres agentes + shadow SDD habilitado.
- **Gap #1 (del audit)**: `DecisionSample` no tiene `agent` / `sessionId` y el MCP server no escribe samples — la atribución por agente aún no es demostrable por construcción.
- **Gate conservador**: comandos seguros pueden devolver `ask` (confianza bajo banda). Si es ruidoso: achicar el matcher a `Bash` o subir la banda de `local_edit` en `policy.ts`.
- **No instalado**: Stop hook y cron-triage (opcionales; requieren scheduler).

10. FAQ rápido

¿Jev elige qué archivos tocar? — Puntúa candidatos que el LLM le propone (`file-scope`), hoy solo en shadow. No inventa ni edita archivos.

¿Jev escribe líneas de código? — Nunca. No genera texto libre.

¿Qué pasa si Jev está caído? — Gate: hard rules siguen negando + el resto pasa a `ask`. Shadow: sample con `error`, nada se bloquea.

¿Cuánto cuesta? — ~$0.00001 por decisión (245 tokens in / 39 out, ~300 ms, medido en vivo).

¿Puedo usarlo sin Jev (offline)? — Sí: `DECISION_ENGINE=rules` degrada a hard-rules-only; el harness sigue siendo utilizable.

Refs: `docs/jev-harness.md` (canonical) · `docs/jev-harness-ledger.md` (metodología) · `docs/jev-harness-usage-audit.md` (audit forense) · `harness/hooks/README.md` · `harness/AGENTS.md`